MERIDIAN – GRAPH-BASED LOGISTICS NETWORK AND ROUTE OPTIMIZATION PLATFORM

A PROJECT REPORT

Submitted by

Saumya Dwivedi (25BCS12109)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

IN

Computer Science Engineering



Chandigarh University

November 2026

Advanced Data Structure • 25CSH-202 • 3rd Semester

Bonafide Certificate



Certified that this project report entitled **"MERIDIAN – GRAPH-BASED LOGISTICS NETWORK AND ROUTE OPTIMIZATION PLATFORM"** is the bonafide work of **Saumya Dwivedi (UID: 25BCS12109; Class: 25BCS-707-B)**, who carried out the project work in partial fulfilment of the requirements for the Bachelor of Engineering in Computer Science Engineering, Chandigarh University.

The report is submitted for academic assessment in **Advanced Data Structure (25CSH-202), 3rd Semester**. The official names, designations, departments, signatures and examination date below are intentionally left blank for completion by the University.

Official field	Entry
Supervisor name / academic designation	_____
Supervisor signature	_____
Department	_____
Head of Department	_____
HOD signature	_____
Examination date	_____

Submitted for the project viva-voce examination held on: _____

Internal Examiner	External Examiner
Signature: _____	Signature: _____
Name/designation: _____	Name/designation: _____

Table of Contents

Bonafide Certificate 2
Table of Contents 3
List of Figures 4
List of Tables 4
List of Standards 4
Abstract 5
Graphical Abstract 6
Abbreviations 7
Symbols 8
CHAPTER 1. INTRODUCTION 9
1.1 Identification of Client / Need / Relevant Contemporary Issue 9
1.2 Identification of Problem 9
1.3 Identification of Tasks 9
1.4 Timeline 9
1.5 Organization of the Report 9
CHAPTER 2. LITERATURE REVIEW / BACKGROUND STUDY 10
2.1 Timeline of the Reported Problem 10
2.2 Existing Solutions 10
Dijkstra with a binary heap 10
Array-based Dijkstra 10
Bellman–Ford 10
2.3 Bibliometric Analysis 11
2.4 Review Summary 11
2.5 Problem Definition 11
2.6 Goals / Objectives 11
CHAPTER 3. DESIGN FLOW / PROCESS 12
3.1 Evaluation and Selection of Specifications / Features 12
3.2 Design Constraints 12
3.3 Analysis of Features and Finalization Subject to Constraints 12
3.4 Design Flow 12
3.5 Design Selection 13
3.6 Implementation Plan / Methodology 13
CHAPTER 4. RESULTS ANALYSIS AND VALIDATION 14
4.1 Implementation of Solution 14
4.1.1 Overall System Architecture 14
4.1.2 Flask Application and API Layer 14
4.1.3 Graph Construction and Representation 14
4.1.4 Database and Data Handling 14
4.1.5 Dijkstra's Binary-Heap Implementation 14
4.1.6 Array-Based Dijkstra Implementation 14
4.1.7 Bellman–Ford Implementation 14
4.1.8 Algorithm Dispatch and Route Reconstruction 15
4.1.9 Network Manager 15
4.1.10 Route Optimizer 15
4.1.11 Comparison Lab 15
4.1.12 Scaling Benchmark 15
4.1.13 Analytics Dashboard 16
4.1.14 User Interface and Application Workflow 16

4.1.15 Testing and Validation 18
4.1.16 Results and Discussion 18
CHAPTER 5. CONCLUSION AND FUTURE WORK 20
5.1 Conclusion 20
5.2 Future Work 20
REFERENCES 21
APPENDIX 22
Appendix 1 – Plagiarism Report 22
Appendix 2 – Design Checklist 22
USER MANUAL 23
Access and basic navigation 23
Calculate and interpret a route 23
Compare, benchmark and inspect the network 23

List of Figures

Figure GA.1: Meridian workflow from logistics need through graph construction, algorithm selection, route optimization and performance analysis 6
Figure 1.1: Indicative project phases; phase order is reconstructed and no dates are asserted 9
Figure 2.1: Directed logistics graph, edge-list and adjacency-list representations, corresponding matrix view, and one relaxation step 10
Figure 3.1: Meridian module architecture and request/data flow 12
Figure 3.2: Algorithm dispatch and path reconstruction flow 13
Figure 4.1: Request-to-result data flow, from UI controls through the API and graph services to SQLite and algorithm outputs ... 14
Figure 4.2: Supplied benchmark time series for heap Dijkstra, array Dijkstra and Bellman–Ford on random and worst-case graphs15
Figure 4.3: Local Standard network Overview screenshot 16
Figure 4.4: Local algorithm comparison view with the three algorithms evaluated on an identical route request 16
Figure 4.5: Local CAI-to-MEX optimizer screenshot, captured after calculation against the isolated local database 17
Figure 4.6: Local scaling benchmark controls and measurements 17
Figure 4.7: Local network manager view; no production records were edited 17
Figure 4.8: Local analytics dashboard for synthetic scenario data 18
Figure 4.9: Local algorithms-explained view 18
Figure 4.10: CAI-to-MEX route segments and direction, with the selected cost objective 19
Figure U.1: Meridian local Overview with the Standard scenario selected 23

List of Tables

Table P.1 – Standards considered as design references 4
Table P.2 – Abbreviations used in this report 7
Table P.3 – Mathematical symbols and notation 8
Table 2.1 – Graph representation trade-offs 10
Table 2.2 – Structured literature comparison 11
Table 3.1 – Feature decision traceability 12
Table 3.2 – Design alternatives and decision 13
Table 4.1 – Reported scaling timings on random sparse graphs (ms) 15
Table 4.2 – Reported scaling timings on Bellman–Ford worst-case graphs (ms) 15
Table 4.3 – Locally reproduced Standard network mode aggregates 16
Table 4.4 – CAI-to-MEX cost-optimal path segment measurements 18
Table 4.5 – Algorithms on the supplied CAI-to-MEX comparison example 19
Table A.1 – Engineering design checklist 22

List of Standards

Standards in this report are design and evaluation references only. The project does not claim certification or demonstrated conformance.

Table P.1 – Standards considered as design references

Standard	Publishing Agency	About the Standard	Page Number
RFC 8259 (2017), The JavaScript Object Notation (JSON) Data Interchange Format	Internet Engineering Task Force (IETF)	Defines JSON syntax and interoperability expectations; relevant to the application's JSON request/response interface. Referenced as a design guide, not a conformance claim.	14

Standard	Publishing Agency	About the Standard	Page Number
ISO/IEC 25010:2023, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model	International Organization for Standardization (ISO) / International Electrotechnical Commission (IEC)	Defines a software-product quality model used as a vocabulary for discussing functional suitability, reliability, performance efficiency, security and maintainability. It was considered for evaluation; no formal SQuaRE assessment was performed.	12

Abstract

Logistics planning can be represented as a shortest-path problem in which facilities and destinations are vertices and transport links are weighted, directed edges. A planner must distinguish the objective being minimized: monetary cost, distance and estimated travel time are not interchangeable. This project, Meridian, implements an interactive Flask application and SQLite-backed synthetic network to demonstrate graph modelling and shortest-path data structures in a reproducible setting.

The backend builds an edge list from persisted routes and creates adjacency lists for traversal. It provides Dijkstra's algorithm with a binary heap, an array-scan Dijkstra variant, and Bellman–Ford. Dijkstra is guarded against negative selected weights; Bellman–Ford supports negative edges and detects negative cycles. The API can dispatch an algorithm, reconstruct predecessor paths, compare implementations, generate benchmark graphs, calculate graph properties and provide network CRUD and analytics. A plain HTML/CSS/JavaScript interface presents the map, optimizer, comparison lab, manager, analytics and algorithm explanations.

On the seeded Standard scenario, the implementation reproduces 25 locations, 102 directed routes, density 0.17, mean out-degree 4.08, one weak component and strong connectivity. For the cost objective, the Cairo-to-Mexico City example returns CAI → MIL → FRA → HAM → NYC → HOU → MEX at $15,616, with separately aggregated distance 16,770 km and estimated time 540.6 h. A local test-client run produced the same route cost for all three algorithms; Dijkstra variants inspected 102 edges and Bellman–Ford inspected 510 in the compared run. The repository test suite passed 25 tests. Timings are environment-dependent; reported sample benchmark values are not universal performance guarantees. The network weights and route data are synthetic, so the project demonstrates algorithm behavior rather than operational logistics performance.

Keywords: weighted directed graph, adjacency list, binary heap, Dijkstra, Bellman–Ford, shortest path, logistics network, Flask, SQLite.

Graphical Abstract

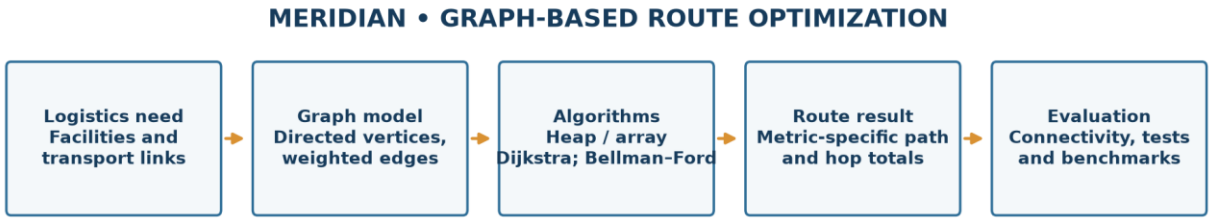


Figure GA.1: Meridian workflow from logistics need through graph construction, algorithm selection, route optimization and performance analysis

The report follows the prescribed five-chapter Chandigarh University structure: problem context and tasks; literature and graph-algorithm background; design choices and methodology; implementation/results/validation; and conclusion/future work.

Abbreviations

Table P.2 – Abbreviations used in this report

Abbreviation	Meaning
API	Application Programming Interface
BF	Bellman–Ford
CRUD	Create, Read, Update and Delete
CSE	Computer Science Engineering
DSU	Disjoint-Set Union
HTTP	Hypertext Transfer Protocol
IETF	Internet Engineering Task Force
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
REST	Representational State Transfer
SCC	Strongly Connected Component
SQL	Structured Query Language
SVG	Scalable Vector Graphics
UID	University Identification Number
URL	Uniform Resource Locator
WSGI	Web Server Gateway Interface

Symbols

Table P.3 – Mathematical symbols and notation

Symbol	Definition
$G = (V, E)$	Graph with vertex set V and directed edge set E
V	Number of vertices (locations)
E	Number of directed edges (routes)
$w(u,v)$	Selected weight of the directed edge $u \to v$
$d(v)$	Current best-known source-to-v distance
$dist(v)$	Final shortest-path distance from the source, when finite
$pred(v)$	Predecessor of v on the current best path
ρ	Directed graph density, $E/[V(V-1)]$ for a loop-free simple digraph
$\delta^+(v)$	Out-degree of vertex v
$\delta^-(v)$	In-degree of vertex v
∞	Unreachable/initial distance sentinel
$-\infty$	No finite minimum for a destination affected by a reachable negative cycle
$O(\cdot)$	Asymptotic upper-bound notation
$\log V$	Logarithm of V; base does not change the asymptotic class

CHAPTER 1. INTRODUCTION

1.1 Identification of Client / Need / Relevant Contemporary Issue

This is an academic demonstration project, not commissioned by a named client. Its motivating issue is the general planning task faced by a logistics network: a shipment may pass through multiple hubs, and choosing a locally attractive link does not necessarily minimize the total journey objective. A graph model makes the decision explicit and permits a route to be evaluated as an ordered sequence of connections. Because transport choices involve trade-offs, Meridian lets the user optimize one edge-weight dimension at a time—cost, distance or estimated time—rather than combining them without a declared policy.

The software is deliberately an educational simulator. It uses reproducibly generated, representative route weights, not carrier quotations, road restrictions, real schedules, customs data or live package tracking. No client interviews, surveys or field measurements were conducted, so the project makes no such claims. Its contemporary relevance is the need to reason transparently about network connectivity, route objectives, data structures and algorithm constraints when building decision-support software.

1.2 Identification of Problem

Given a loop-free directed weighted graph G = (V, E), a source s, destination t and a selected non-negative edge-weight function w, find a path from s to t whose total weight is minimum. If no path exists, report that the destination is unreachable. The total must be based on the selected metric, not on visual distance alone. A second problem is to determine when the assumptions of a chosen algorithm are not satisfied: Dijkstra is not valid for negative edge weights, while a reachable negative-weight cycle can make the minimum undefined for affected destinations.

The implementation problem also includes preserving enough predecessor information to explain a result, providing comparable execution counters, and communicating graph properties to a user. These are constraints on a valid solution, not claims about a production transport network.

For a path $P = (v_0, \ldots, v_k)$, its selected weight is:

$$\mathbf{W(P) = \Sigma_{i=0}^{k-1} w(v_i, v_{i+1}), \text{ and } dist_s(t) = \min_P : s \to t \; W(P).}$$

If no s→t path exists, $dist_s(t)$ is ∞. If a cycle of negative total weight is reachable from s and can reach t, there is no finite minimum for t.

1.3 Identification of Tasks

The project tasks were to: (1) inspect and model a directed logistics graph with distinct cost, distance and time weights; (2) persist locations and routes in SQLite; (3) implement heap Dijkstra, array Dijkstra and Bellman–Ford with path predecessors; (4) validate non-negative-weight preconditions and negative-cycle behavior; (5) expose algorithms and management operations through a Flask JSON API; (6) build a browser UI for selection, visualization, comparison and analytics; (7) generate deterministic sample and benchmark graphs; and (8) test algorithms, API behavior and representative results. These tasks map to a single implementation chapter rather than separate chapters for architecture, algorithms or interface.

1.4 Timeline

No dated project log was available in the repository. The following is an indicative work sequence reconstructed from project deliverables, not a claim about actual calendar dates.

Phase	Work package	Evidence / output
1	Define problem and graph model	Vertices, directed routes, selected weight semantics
2	Implement data and algorithms	SQLite schema, three shortest-path implementations
3	Integrate application	Flask endpoints, services and single-page interface
4	Evaluate and document	Automated tests, benchmark cases, analytics and report

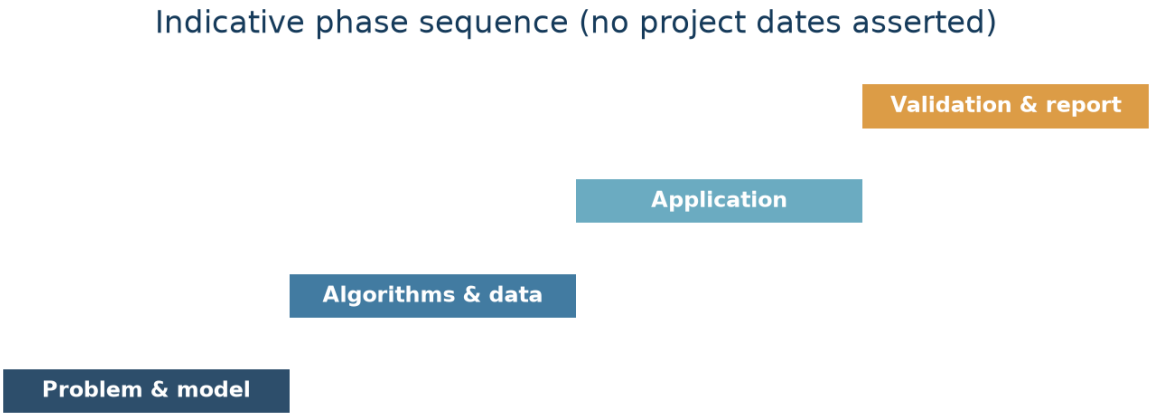


Figure 1.1: Indicative project phases; phase order is reconstructed and no dates are asserted

1.5 Organization of the Report

Chapter 1 establishes the need, solution-independent problem, tasks and indicative work sequence. Chapter 2 reviews graph representations, shortest-path methods, existing approaches and the academic problem/objectives. Chapter 3 compares design alternatives and describes the selected flow and implementation methodology. Chapter 4 presents the actual Flask, SQLite, graph, algorithm and UI implementation, followed by testing, results and limitations. Chapter 5 summarizes achievements and proposes future work. References, appendices and the User Manual follow the five chapters as in the local University format.

CHAPTER 2. LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Reported Problem

The shortest-path problem arises in routing, transportation and communication networks. Its formal graph treatment predates modern web applications. Dijkstra published a label-setting shortest-path method in 1959; Bellman's work on routing in 1958 is an early source for repeated edge relaxation. These methods remain complementary: Dijkstra exploits non-negative weights, while Bellman–Ford handles negative weights and detects negative cycles. Later data-structures texts systematized priority queues, adjacency representations and complexity analysis. Contemporary software commonly combines these classical algorithms with persistent data, APIs and interactive presentation; Meridian is an instructional integration of those pieces rather than a novel shortest-path algorithm [1]–[4].

2.2 Existing Solutions

For a directed graph G = (V,E), an adjacency list stores each vertex's outgoing neighbours and weights; an edge list stores tuples (u,v,w); an adjacency matrix stores a V×V weight table. A list uses Θ(V+E) memory and supports neighbour iteration in Θ(out-degree(u)); a matrix uses Θ(V²) memory and provides constant-time edge lookup at the cost of scanning a full row for neighbours. The implementation uses both an edge list (database-derived routes) and a dictionary-backed adjacency list when algorithms traverse edges. The matrix is a theoretical comparison, not a live implementation.

Table 2.1 – Graph representation trade-offs

Representation	Storage	Edge lookup	Neighbour traversal	Typical fit
Edge list	Θ(V+E)	Θ(E) without indexing	Θ(E) for all outgoing edges	Edge-relaxation algorithms
Adjacency list	Θ(V+E)	Θ(out-degree) in this list-based form	Θ(out-degree)	Sparse graphs and traversals
Adjacency matrix	Θ(V²)	Θ(1)	Θ(V) per vertex	Dense graphs / frequent direct lookup

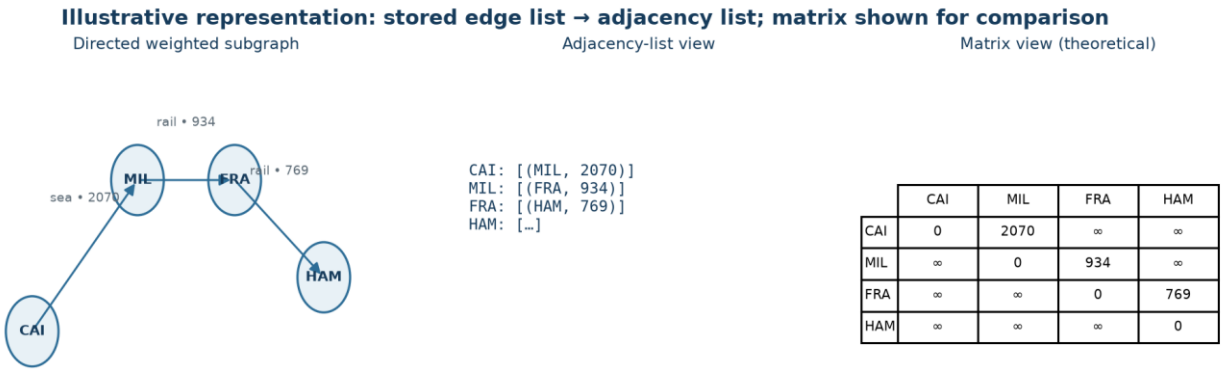


Figure 2.1: Directed logistics graph, edge-list and adjacency-list representations, corresponding matrix view, and one relaxation step

Let n = |V| and m = |E|. For a loop-free simple directed graph, $0 \le m \le n(n-1)$, and density is $\rho = m/[n(n-1)]$. In the seeded Standard case, ρ = 102/(25×24)=0.17. Average out-degree is $m/n = 4.08$. In-degree counts incoming directed edges; out-degree counts outgoing ones. A weakly connected component ignores edge direction for reachability, whereas a strongly connected component requires directed paths both ways between every pair in the component. One weak component does not imply strong connectivity; the seeded graph separately has one SCC and is therefore strongly connected.

Dijkstra with a binary heap

Dijkstra is greedy: it repeatedly settles the unsettled vertex with the smallest tentative distance. Initialize d(s)=0 and d(v)=∞ otherwise; insert (0,s) into a min-priority queue. On extracting the smallest key, skip a stale duplicate or an already-settled vertex, then examine outgoing edges. Relax u→v when d(u)+w(u,v)<d(v); set d(v) and pred(v), and push the new key. The heap root is the current minimum, and push/pop cost O(log V) under the simple-graph assumptions. The implementation uses Python's min-heap and lazy duplicate entries rather than decrease-key. With adjacency lists, the common bound is O((V+E)log V); storage including graph, predecessor/distance maps and heap is O(V+E). Non-negative weights are essential to the greedy settling argument [1], [3], [5].

Array-based Dijkstra

The array variant maintains tentative distances and repeatedly scans remaining vertices to find the minimum. Its minimum selection costs O(V) per settled vertex, for O(V²) selection work, plus O(E) relaxation work: O(V²+E), often stated as O(V²). The implementation still uses the adjacency list for outgoing edges, so storage remains O(V+E), not O(V²). On sparse graphs the repeated full scans can dominate; on dense graphs the O(V²) scan can be competitive with heap operations, with practical crossover depending on representation, runtime and workload. It also requires non-negative weights.

Bellman–Ford

Bellman–Ford initializes distances similarly and scans every directed edge repeatedly. Each relaxation tests **d(v) > d(u)+w(u,v)**; after V−1 complete passes, all finite shortest paths (which can be made simple absent negative cycles) have been considered. If a pass makes no change, the implementation stops early. A further pass detects a reachable negative cycle. The source-based version uses O(VE) worst-case time and O(V+E) space; a virtual-source mode initializes all distances to zero to detect cycles anywhere in the graph. The code records predecessors, extracts a cycle, computes affected vertices and marks them −∞. This generality costs repeated full edge scans; it is not automatically preferable when all weights are non-negative [2]–[4].

2.3 Bibliometric Analysis

No systematic database search, citation export or bibliometric corpus was available, so publication counts, citation ranks and year-by-year bibliometric statistics are not asserted. A structured comparative review is more defensible for this project: it relates seminal shortest-path work and authoritative data-structures references to concrete implementation choices.

Table 2.2 – Structured literature comparison

Source / approach	Key contribution used here	Strength	Limitation relevant to this project
Dijkstra (1959) [1]	Greedy label-setting shortest paths	Efficient when all selected weights are non-negative	Does not support negative weights
Bellman (1958) [2]	Routing formulated through iterative relaxation	Establishes a general relaxation perspective	Repeated passes can be expensive
Ford (1956) [3]	Network-flow / shortest-path foundations	Connects network problems to algorithmic optimization	Historical report; not a modern implementation guide
Cormen et al. [4]	Graph representations, heaps and complexity	Precise algorithmic reasoning and bounds	Abstract models do not predict Python wall-clock time
Python `heapq` documentation [5]	Min-heap operations used by implementation	Language-specific operational behavior	API documents data structure, not Dijkstra correctness

The useful comparison is qualitative and algorithmic, not a numerical publication survey. The project evaluates whether representation and preconditions match the modeled task and then tests implementation behavior.

2.4 Review Summary

The reviewed methods establish a clear selection rule. Use a heap-based Dijkstra implementation when the selected objective has only non-negative weights and the graph is sparse. An array scan is a reasonable alternative for smaller or sufficiently dense graphs. Use Bellman–Ford if negative weights are permitted and a finite shortest path remains well-defined; detect negative cycles because no finite optimum exists for affected destinations. Adjacency lists suit the Standard network's 102 edges among 25 vertices and support local neighbour traversal. The edge list remains convenient for persistence and Bellman–Ford's repeated relaxation. The reviewed literature informs this design but does not make Meridian's synthetic weights operational evidence.

2.5 Problem Definition

For a persisted network, selected metric $q \in \{\text{cost}, \text{distance}, \text{time}\}$, source s and target t, compute a minimum-weight directed path using **wq(u,v)** from the corresponding route field. Return the ordered vertices, predecessor-derived hop sequence, selected total, and distance/cost/time totals for interpretation. Reject invalid nodes, metrics and algorithms; prevent Dijkstra on negative selected weights; distinguish unreachable targets from negative-cycle-affected targets. For comparison, execute valid algorithm variants on identical graph data and expose complexity metadata, edge checks and runtime observations. The scope excludes real-time network feeds, multi-objective optimization, capacities, inventory allocation and proof of real delivery outcomes.

2.6 Goals / Objectives

1. Model locations and directed transport links as G=(V,E), keeping cost, distance and time as distinct edge weights.
2. Implement and compare heap Dijkstra, array Dijkstra and Bellman–Ford with predecessor-based path reconstruction.
3. Enforce algorithm preconditions and report unreachable destinations and negative-cycle effects without presenting an invalid finite optimum.
4. Persist reproducible scenarios and expose route, analysis, comparison, benchmark, analytics and network-management operations through Flask/SQLite.
5. Validate algorithms using known cases, generated cross-check graphs and API tests; interpret measured timings as environment-dependent observations.
6. Deliver a UI that communicates the selected metric and algorithm result, while identifying the synthetic nature and limitations of the data.

CHAPTER 3. DESIGN FLOW / PROCESS

3.1 Evaluation and Selection of Specifications / Features

The essential features were selected to answer graph-algorithm questions rather than to maximize interface breadth: persistent directed locations/routes; a user-selected weight; three comparable shortest-path algorithms; path details and counters; negative-edge/cycle handling; reproducible scenarios and benchmark generators; and a visual interface for route selection and network properties. Optional management and analytics functions support experimentation by making the graph inspectable and editable in a local simulator. The JSON boundary follows the interoperable data-exchange concepts of RFC 8259 [8], without claiming a formal conformance audit.

The implementation requirements are: (a) preserve edge direction, (b) keep the three objectives separate, (c) store enough metadata to explain each hop, (d) validate endpoint and weight constraints, (e) expose algorithm limitations, and (f) make tests repeatable. This corresponds to the functional-suitability and performance-efficiency vocabulary of ISO/IEC 25010:2023, used only as a design reference [9].

3.2 Design Constraints

Technical constraints. The app uses Python, Flask, SQLite and plain browser technologies. No external map or chart service is required. The map is a stylized projection, not a geographic routing engine. SQLite uniqueness means parallel edges with the same ordered endpoints are not represented. Graphs are loaded into memory before shortest-path calculation.

Algorithmic constraints. Dijkstra requires non-negative weights for the selected metric, even if another metric on the same route is non-negative. Bellman–Ford can handle negative edges but cannot produce a finite minimum to vertices affected by a reachable negative cycle. The standard graph is directed; two-way corridors are two directed records. Negative cost credits exist only in testing scenarios.

Data and evaluation constraints. The generated transport model estimates route measures from coordinates and fixed mode constants. It omits schedules, congestion, capacity, reliability, customs and live prices. Benchmark timings depend on Python/runtime/hardware and graph ordering; hardware metadata for the supplied screen values is unavailable. The application does not implement authentication or claim production security certification.

3.3 Analysis of Features and Finalization Subject to Constraints

Table 3.1 – Feature decision traceability

Requirement	Chosen feature	Verification evidence
Explain a route	Predecessor map, route reconstruction and hop rows	Algorithm tests and route API
Compare costs, time and distance	Metric selector; distinct weights per edge	`GraphData.edges(metric)`
Handle negative testing weights safely	Dijkstra precondition check and Bellman–Ford cycle detection	Algorithm/API tests
Demonstrate graph-scale trends	Deterministic random and BF worst-case graph generators	Benchmark tests and UI
Support inspectable model data	SQLite-backed network/location/route CRUD	API validation tests
Communicate connectivity	Density, degree, weak and strong components	Graph analysis and dashboard

Two route alternatives were considered. A matrix-oriented design would use $\Theta(V^2)$ memory and constant-time edge lookup, then array-scan Dijkstra in $O(V^2+E)$; it may be attractive for dense small graphs but wastes space on sparse networks. The selected sparse design stores edges and builds adjacency lists, using heap Dijkstra with $O((V+E)\log V)$ expected textbook work under non-negative weights. Bellman–Ford remains an implemented alternative when negative weights require general relaxation. A third conceptual production direction—A* or preprocessed routing—was not implemented because no heuristic/real geographic road graph or large operational workload is present.

3.4 Design Flow

The flow begins with a request carrying network ID, source, target, algorithm and metric. The API validates the JSON body and delegates to services. Services load the network's vertices and route rows, map the metric to an edge weight, compute graph properties/recommendation, run the chosen algorithm, reconstruct a path and serialize result/diagnostic data. Flask returns JSON; the front end renders map, route details, charts and messages. Analytics use stored routes and recent run records, while benchmark endpoints construct generated graphs independently.

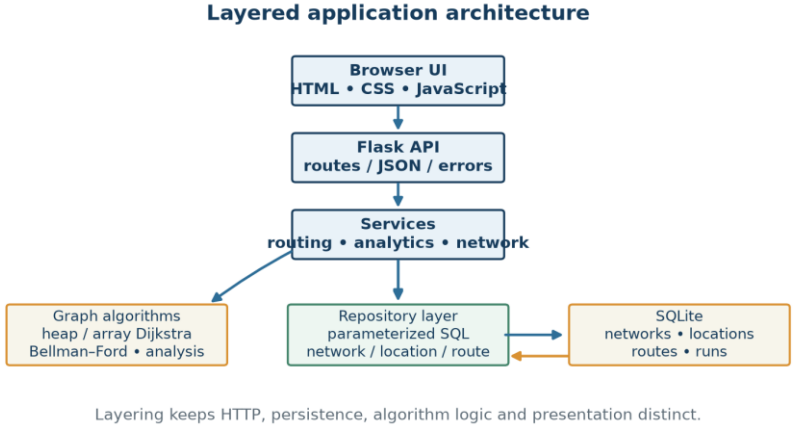


Figure 3.1: Meridian module architecture and request/data flow

The route path reported by the backend, not the SVG map geometry, is authoritative. The model separates stored route data, shortest-path computation, API transport and presentation. Error objects cross the service boundary with status and code; unexpected exceptions are logged by the application factory.

3.5 Design Selection

The selected design keeps pure algorithm functions in `backend/algorithms`, persistence in `backend/db`, orchestration in `backend/services`, HTTP parsing in `backend/routes`, and UI concerns in `frontend`. This separation enables textbook cases to test the algorithms without Flask or SQLite and enables API tests to exercise validation and serialization. A relational schema enforces local integrity constraints; Python structures are derived at request time and discarded after calculation.

Table 3.2 – Design alternatives and decision

Alternative	Advantages	Costs / restrictions	Status
Adjacency list + heap Dijkstra	O(V+E) graph storage; efficient sparse traversal	Heap operations; invalid for negative weights	Implemented and recommended on sparse non-negative graphs
Matrix + array Dijkstra	Simple selection; dense-graph scan can be competitive	$\Theta(V^2)$ memory and $O(V^2)$ scan even for sparse graphs	Conceptual comparison; not the repository representation
Edge-list Bellman–Ford	Negative-edge support and negative-cycle detection	O(VE) worst-case repeated scans	Implemented; selected when negative edges require it
Real-time road routing service	Could reflect live constraints if integrated	Requires external data/service and operational controls	Not implemented; future option only

The recommender does not benchmark every possible implementation before every route. It derives a rule from metric-specific negative weights, graph density and size. This is a deterministic heuristic, not a proof of universal runtime optimality.

3.6 Implementation Plan / Methodology

Implementation proceeds from schema and reproducible sample data to pure algorithms, then graph-property analysis, route services and API endpoints, followed by browser views. Validation uses known examples, negative-cycle and unreachable cases, random positive graphs cross-checked across all three algorithms, and API tests for scenarios, CRUD, errors and benchmarks. Benchmarks report medians and operation counts for generated cases; correctness checks remain separate from timing claims.

The resulting data flow is summarized below. A source/target request is resolved against graph nodes; metric selection maps to `cost_usd`, `distance_km` or `time_hours`; a negative cycle or unreachable target changes the status instead of returning a fabricated path.

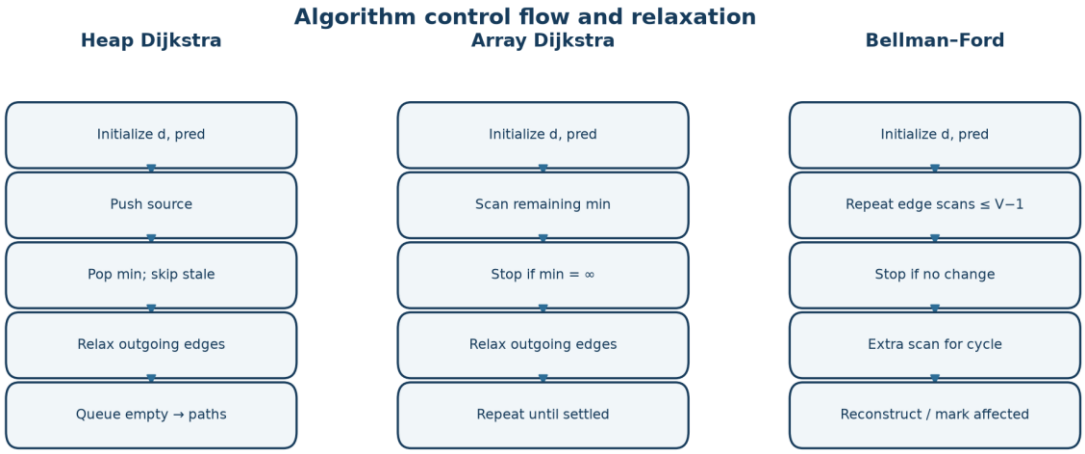


Figure 3.2: Algorithm dispatch and path reconstruction flow

The design is intentionally small enough for a third-semester data-structures project: it demonstrates representation trade-offs, priority-queue versus scan-based minimum selection, repeated edge relaxation, predecessor tracking and empirical complexity trends without representing its synthetic input as real operations data.

CHAPTER 4. RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

4.1.1 Overall System Architecture

Meridian is a layered Flask application. `run.py` invokes `create_app`; the factory initializes the schema/seeds missing scenarios, registers the API blueprint and serves `frontend/index.html` plus static assets. API routes call service functions, which read SQLite through a repository and invoke pure graph algorithms. Browser JavaScript calls JSON endpoints, then renders the active view. This is a single-page interface, not a separate frontend build. The local project tests and sample screenshots use an isolated database under the report output; the existing application database was not changed.

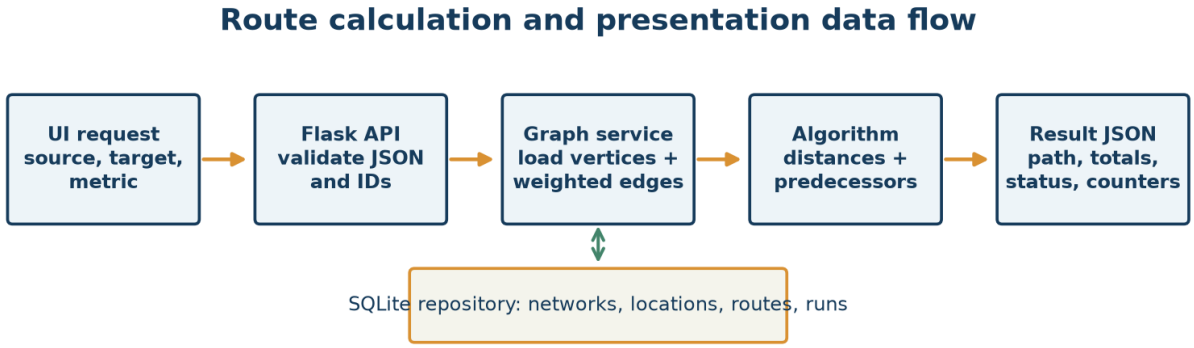


Figure 4.1: Request-to-result data flow, from UI controls through the API and graph services to SQLite and algorithm outputs

4.1.2 Flask Application and API Layer

`backend/app.py::create_app` constructs Flask, configures the database path and maps API errors to JSON. `backend/routes/api.py` defines endpoints for health/theory, network list/generation/reset/delete, graph/summary/analysis/analytics, route, compare, benchmark and location/route CRUD. Request bodies must be JSON objects for the main APIs. `ApiError` carries an HTTP status and code; malformed arguments, unknown resources, duplicates and invalid weights are reported with suitable 400/404/409/422 responses. Unexpected exceptions are logged and returned as generic 500 errors.

4.1.3 Graph Construction and Representation

The repository returns locations and directed route rows. `GraphData` stores a node-code list, a code-to-location dictionary and an ordered-pair route map. For a selected metric, `GraphData.edges` produces `(src,dst,weight)` tuples. `common.py::build_adjacency` creates `dict[node] -> list[(neighbour,weight)]`. This combination keeps route rows convenient for persistence and edge-relaxation while supporting outgoing-neighbour traversal for Dijkstra. No adjacency matrix is used by the running algorithm code.

The density is `E/[V(V-1)]` for this no-self-loop graph; average out-degree is E/V. `weak_components` uses disjoint sets with direction ignored. `strong_components` uses iterative Kosaraju on the forward and reverse adjacency structures. Each property answers a different structural question: density describes possible directed links occupied, average degree describes mean outgoing options, weak connectivity ignores orientation and SCC count respects orientation fully.

4.1.4 Database and Data Handling

`database.py` declares four tables: `networks`, `locations`, `routes` and `runs`. Primary keys, foreign keys, uniqueness, CHECK constraints and network indexes support integrity and lookups. A route is directed; bidirectional creation inserts two rows. Location deletion removes incident route rows in repository logic. `runs` records algorithm, metric, endpoints, status, total, time, relaxations and iterations. SQLite is the project's single-process demo persistence layer; no distributed database, authentication system or live carrier integration is claimed.

4.1.5 Dijkstra's Binary-Heap Implementation

`algorithms/dijkstra.py::dijkstra(..., variant="heap")` rejects negative selected weights by default, builds adjacency lists and initializes `dist[source]=0`, all other distances to infinity, and all predecessors to `None`. It pushes `(distance,node)` pairs into `heapq`. The smallest tentative key is extracted; stale entries are discarded using a settled set. For each outgoing edge, the algorithm counts an edge check, computes the candidate distance and relaxes a strictly shorter label. The updated predecessor is retained for path reconstruction. Lazy insertion can place duplicate keys in the heap; it avoids an explicit decrease-key implementation. For simple graphs, this yields the common O((V+E)logV) time bound and O(V+E) total storage. It is valid only for non-negative weights.

4.1.6 Array-Based Dijkstra Implementation

The same function's `variant="array"` maintains remaining vertices and obtains the minimum with a linear `min(remaining, key=dist.__getitem__)` scan. The implementation counts each scan's remaining-set size, then relaxes outgoing adjacency entries. Worst-case selection is $O(V^2)$, edge work O(E), hence $O(V^2+E)$; the graph and distance structures still occupy O(V+E) space. For an unreachable remainder, it stops as soon as the minimum distance is infinite. Its validity condition is the same as heap Dijkstra's.

4.1.7 Bellman–Ford Implementation

`bellman_ford.py::bellman_ford` works over the edge list. With a specified source, it performs at most V−1 relaxation passes; with `source=None`, all distances start at zero and V passes are allowed for global cycle analysis. A no-change pass sets `early_termination` and stops. If all passes were needed, a further scan checks whether an edge can still improve a reachable distance. Predecessors are used to walk into and extract a cycle; affected downstream vertices are found by graph traversal and assigned $-\infty$. The worst-case time is O(VE) plus a linear detection scan, and aggregate graph/distance/predecessor storage is O(V+E).

4.1.8 Algorithm Dispatch and Route Reconstruction

`services/graph.py::run_algorithm` maps `dijkstra_heap`, `dijkstra_array` and `bellman_ford` to implementations. `auto` in route calculation first asks `recommend`; negative selected edges lead to Bellman–Ford, a negative cycle is surfaced as invalid/no finite optimum, and non-negative data leads to a Dijkstra variant. Dijkstra errors are converted to a 422 response with a recommendation. `ShortestPathResult.path_to` follows predecessor links back to the source and refuses unreachable or negative-infinity destinations. `describe_path` looks up each directed edge and sums all three measures independently. Optional alternatives remove one best-path link at a time and re-solve; these are single-link-failure alternatives, not an exhaustive k-shortest-path algorithm.

4.1.9 Network Manager

The network service validates location codes, kinds, coordinates and package counts; route endpoints, mode, distance, cost, time and weekly trips; and duplicate rows. Route creation can calculate omitted measures using coordinates and mode constants; a bidirectional option inserts a reverse edge. The UI/API supports edit and removal; removing a location removes its incident routes. Built-in scenarios can be reset but not deleted; generated networks can be deleted. Negative synthetic cost values can be entered for algorithm demonstrations. No production transport record was changed for this report.

4.1.10 Route Optimizer

The interface offers network scenario, metric, source, target, algorithm and alternative-route controls. Overview map points can select endpoints; the optimizer then calls the route API and presents the path, total for the selected metric, a hop table, counters, runtime and alternatives where available. The map is a coordinate projection for orientation, while the path is calculated from stored edges. A selected cost path does not imply minimum distance or minimum time; all three total values are shown to make that distinction explicit.

4.1.11 Comparison Lab

The comparison service runs both Dijkstra variants and Bellman–Ford against identical graph data, reports applicable status and complexity metadata, and calculates repeated-run medians (repeat count is bounded). For negative weights, Dijkstra is normally marked inapplicable; a forced mode exists to demonstrate incorrect behavior, not to recommend it. Agreement indicates matching totals among applicable, unforced algorithms for that instance; it is not a proof for every graph.

4.1.12 Scaling Benchmark

`routing.py::benchmark` accepts sizes from 3 to 2,000, repeats and average degree. It compares all three methods on deterministic positive random graphs and on a reverse-ordered chain with additional edges that forces repeated Bellman–Ford passes. Measurements use `perf_counter` and medians; the source and endpoint document that timing values vary by machine and runtime.

The following values were supplied as displayed benchmark evidence in the project brief; the source screenshots themselves were not accessible locally for direct pixel-level verification. They are reported as such, not as fresh controlled measurements from this machine.

Table 4.1 – Reported scaling timings on random sparse graphs (ms)

Vertices	Heap Dijkstra	Array Dijkstra	Bellman–Ford
25	0.044	0.055	0.152
50	0.081	0.189	0.263
100	0.184	0.402	0.532
200	0.494	1.259	1.655
400	0.941	4.931	2.665
800	1.728	19.645	7.924

Table 4.2 – Reported scaling timings on Bellman–Ford worst-case graphs (ms)

Vertices	Heap Dijkstra	Array Dijkstra	Bellman–Ford
25	0.081	0.098	0.290
50	0.080	0.129	1.146
100	0.167	0.392	4.773
200	0.352	1.261	20.169
400	0.742	5.417	84.941
800	1.677	19.827	343.631

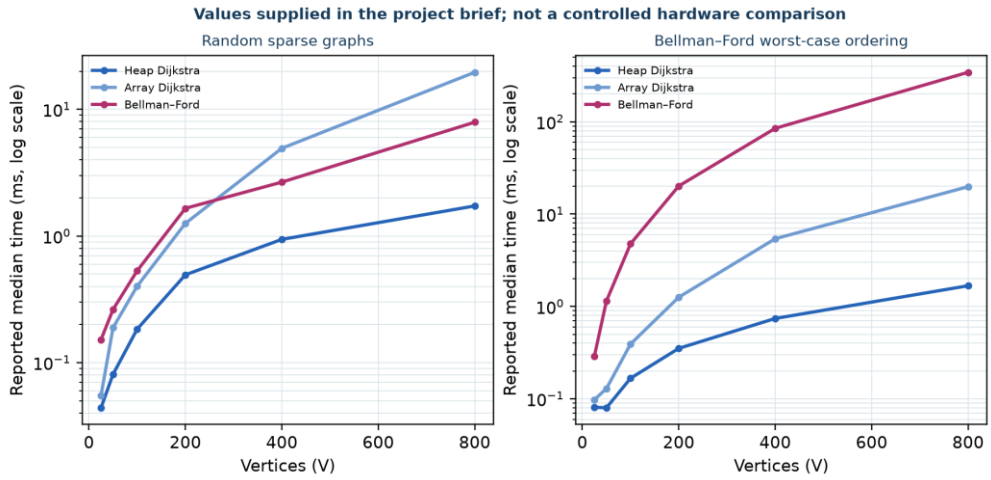


Figure 4.2: Supplied benchmark time series for heap Dijkstra, array Dijkstra and Bellman–Ford on random and worst-case graphs

On sparse graphs the array's repeated minimum scans grow quadratically in V, while heap Dijkstra avoids scanning every remaining vertex. Bellman–Ford's random-graph values benefit from early termination; they are not its worst-case bound. In the reverse-ordered

graph, edge order delays propagation, producing many passes and a marked increase consistent with O(VE). The graph generator's edge counts and ordering are part of the experimental condition. A single time series does not establish universal performance superiority; hardware/runtime, warm-up, repeats, graph shape and system load matter.

4.1.13 Analytics Dashboard

The Standard scenario analytics are computed from generated rows. Weekly spend is the sum of each directed route's synthetic cost multiplied by weekly trips. Mode average delivery time is the arithmetic mean of route times within that mode; cost/km is aggregate route cost divided by aggregate distance. A low aggregate sea cost/km does not mean sea is fastest: the supplied metrics show the converse trade-off. Hubs are ranked by combined in- and out-degree, with packages shown as stored static counts. Route efficiency compares a direct edge with a calculated best-path total for that source/target; the statistic is metric-specific. Shortest-delivery averages derive from graph paths and should not be read as measured shipment telemetry.

Table 4.3 – Locally reproduced Standard network mode aggregates

Mode	Weekly spending (USD)	Average route time (h)	Aggregate cost/km (USD/km)
Air	8,686,620	14.5	6.870
Rail	164,290	33.6	1.293
Road	826,000	25.6	1.907
Sea	431,076	276.4	0.467

Rounded individual spending figures need not sum to an independently rounded total. Analytics also reports graph components and algorithm runs; the run table is empty until route calculations are recorded.

4.1.14 User Interface and Application Workflow

The front end is served from `frontend/index.html` and is implemented in plain HTML, CSS and JavaScript. Sidebar controls select scenario and metric; navigation switches among Overview, optimizer, comparison, manager, analytics and algorithm explanations. `map.js` draws the network and route as SVG; `charts.js` draws charts without external chart libraries. App state and API calls are managed by `app.js`; request failures appear through a status toast. The deployed URL rendered its Overview after a cold start. Screenshots in this report are labelled local UI captures; they do not prove a specific deployed build's exact commit.

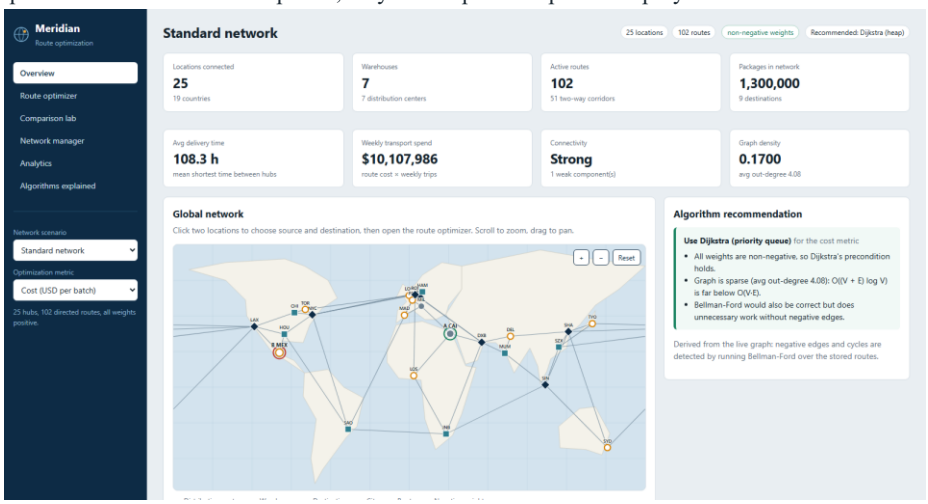


Figure 4.3: Local Standard network Overview screenshot

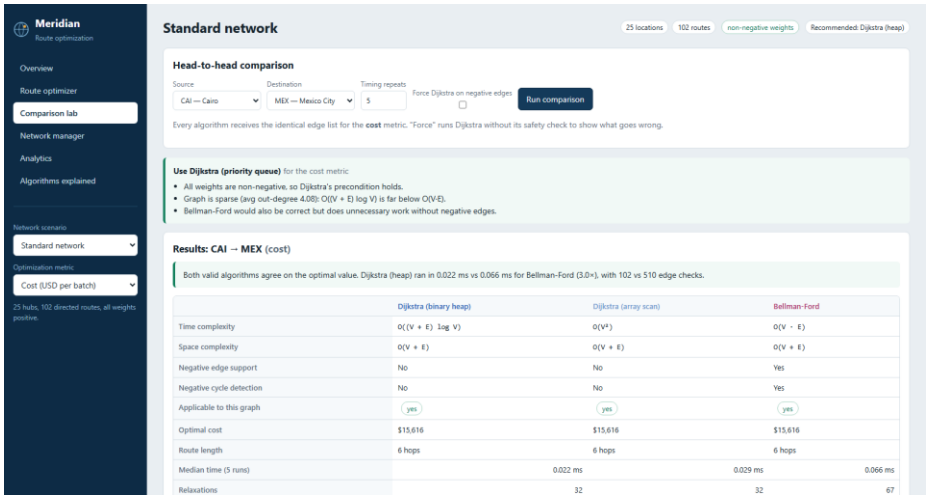


Figure 4.4: Local algorithm comparison view with the three algorithms evaluated on an identical route request

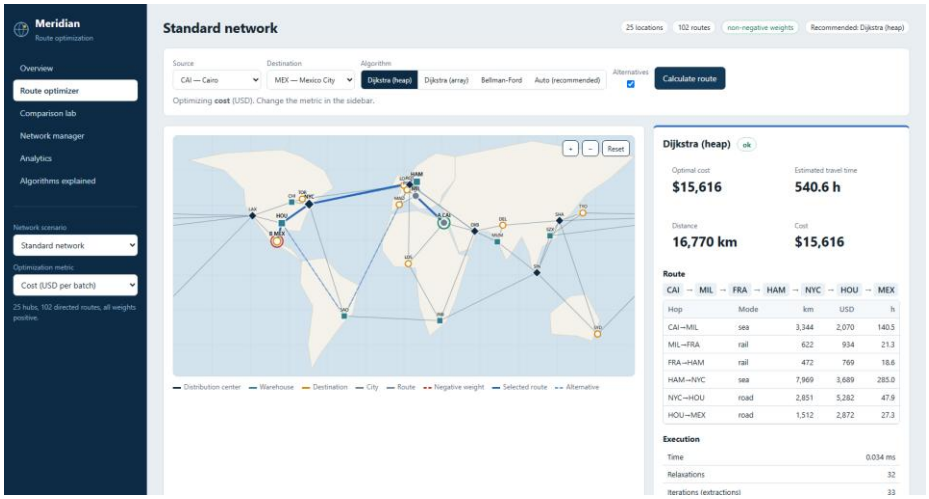


Figure 4.5: Local CAI-to-MEX optimizer screenshot, captured after calculation against the isolated local database

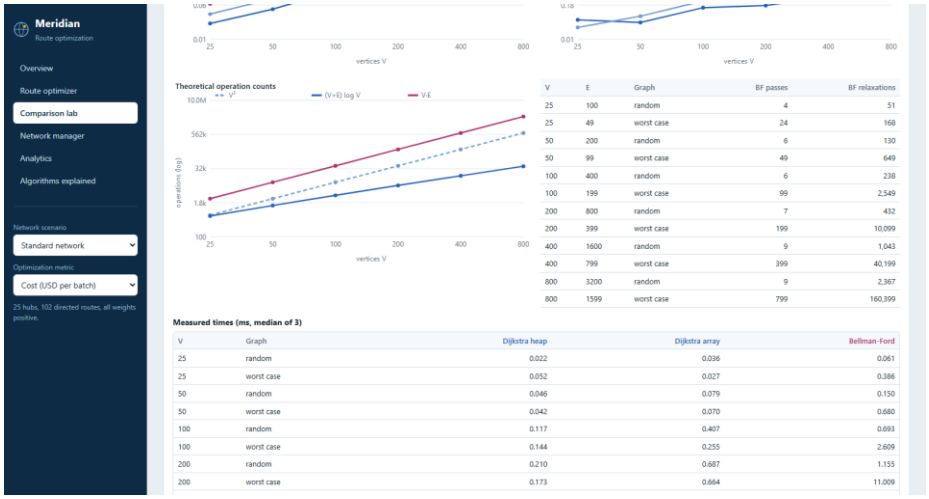


Figure 4.6: Local scaling benchmark controls and measurements

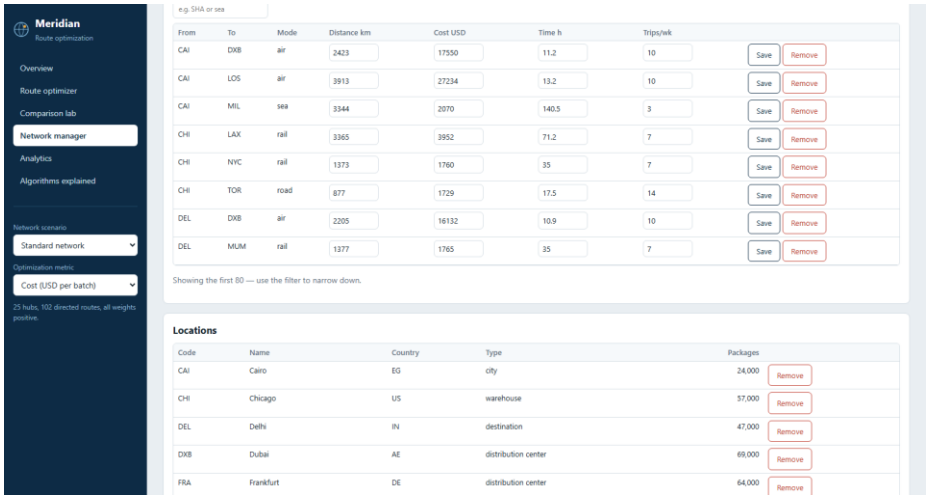


Figure 4.7: Local network manager view; no production records were edited

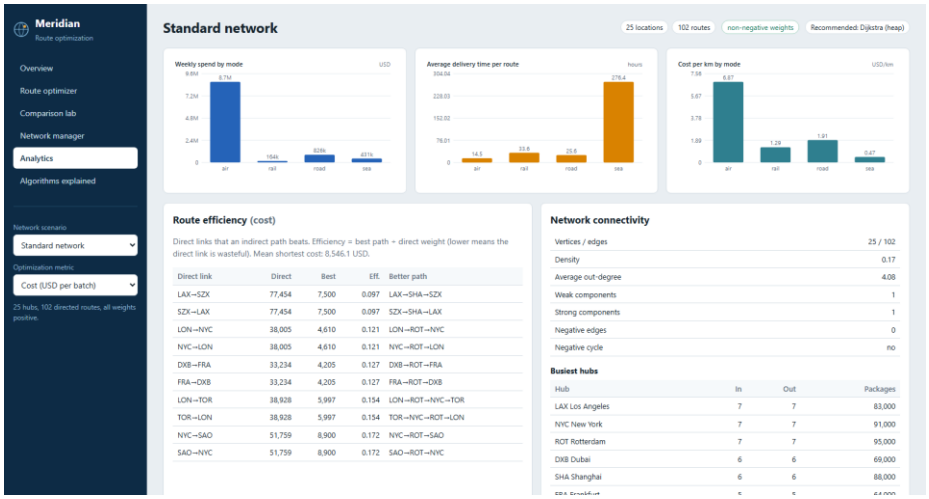


Figure 4.8: Local analytics dashboard for synthetic scenario data

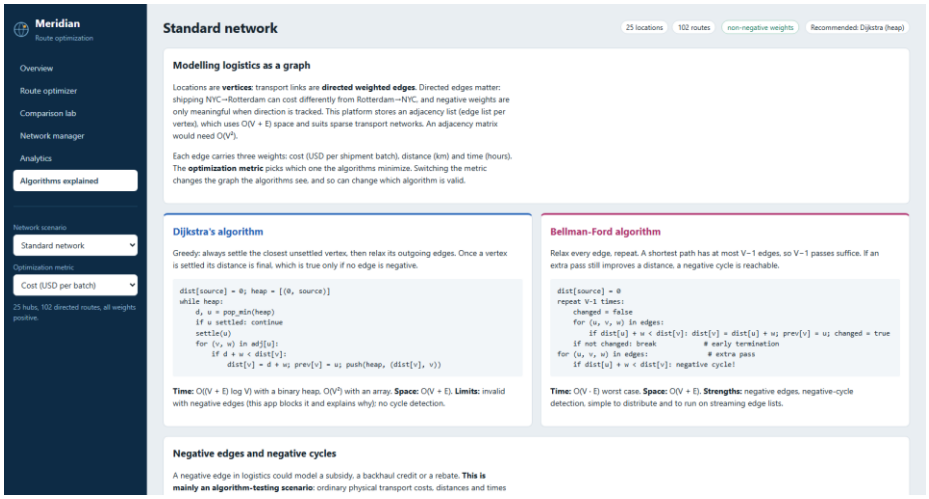


Figure 4.9: Local algorithms-explained view

4.1.15 Testing and Validation

The application test environment reported **25 passed**. Algorithm tests include textbook non-negative/negative examples, unknown and unreachable nodes, Dijkstra negative-weight rejection, negative-cycle extraction, unreachable-cycle behavior, early termination, a reverse-order worst case, 40 generated cross-check graphs, and a forced-Dijkstra counterexample. API tests cover four scenarios, route equivalence, negative cases and recommendations, comparison, benchmarking, analytics, CRUD and validation errors. The test suite is automated; no human-subject study, user survey, production load test or security audit was performed.

Local test-client verification reproduced the Standard counts, graph properties, analytics aggregates and CAI-to-MEX route. A fresh comparison run returned the same $15,616 total for all three algorithms and 102 Dijkstra edge checks versus 510 for Bellman–Ford. A representative local median sample was 0.0209 ms heap, 0.0280 ms array and 0.0633 ms Bellman–Ford, different from the prompt-supplied screen values of 0.086/0.116/0.264 ms. This difference is expected for wall-clock microbenchmarks and reinforces that such values depend on measurement environment and repeat protocol.

4.1.16 Results and Discussion

The Cairo-to-Mexico route contains six directed edges. Each segment's measures are shown separately below; the reported route total is the sum of its edges.

Table 4.4 – CAI-to-MEX cost-optimal path segment measurements

Segment	Mode	Distance (km)	Cost (USD)	Time (h)
CAI → MIL	Sea	3,344	2,070	140.5
MIL → FRA	Rail	622	934	21.3
FRA → HAM	Rail	472	769	18.6
HAM → NYC	Sea	7,969	3,689	285.0
NYC → HOU	Road	2,851	5,282	47.9
HOU → MEX	Road	1,512	2,872	27.3
Total	**6 hops**	**16,770**	**15,616**	**540.6**

The path uses sea links for lower-cost long-haul segments, rail for European connections and road for regional links in the example. This interpretation describes the synthetic weights only; it is not a recommendation for actual shipping.



Figure 4.10: CAI-to-MEX route segments and direction, with the selected cost objective

Table 4.5 – Algorithms on the supplied CAI-to-MEX comparison example

Measure	Heap Dijkstra	Array Dijkstra	Bellman–Ford
Time complexity	O((V+E)logV)	$O(V^2+E)$	O(VE)
Aggregate space	O(V+E)	O(V+E)	O(V+E)
Negative weights	No	No	Yes
Negative-cycle detection	No	No	Yes
Selected cost	$15,616	$15,616	$15,616
Hops	6	6	6
Supplied displayed median (ms)	0.086	0.116	0.264
Edge checks in supplied display	102	102	510

Equal route costs are expected because this Standard example has non-negative selected weights, so all three algorithms compute the same shortest-path objective. The 510 Bellman–Ford checks correspond to five complete scans of 102 edges (four relaxation passes plus the additional check); the Dijkstra variants inspect the outgoing edges of settled vertices, producing 102 checks here. The reported medians are from the supplied project evidence description; local remeasurement differed and is documented above.

The result supports the implementation goals: the application models a directed weighted graph, exposes three algorithmic strategies, blocks invalid Dijkstra use, reconstructs routes and demonstrates benchmark growth. It does not show real logistics savings or prove a universal speed ranking. Important limitations include synthetic weights, no schedule/time-dependent routing, no multi-objective optimization, in-memory graph construction per request, a stylized map, no live data integration and no formal security/conformance assessment. These boundaries constrain interpretation of all reported results.

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 Conclusion

Meridian demonstrates how a logistics network can be modelled as a directed weighted graph and explored through complementary shortest-path data structures. The active representation combines database-derived edge tuples with dictionary-backed adjacency lists. Heap Dijkstra uses a priority queue and is appropriate for sparse, non-negative selected weights; the array variant shows the cost of linear minimum selection; Bellman–Ford supports negative edges and reports negative cycles. Predecessor tracking turns distances into explainable paths.

The Flask/SQLite application integrates these algorithms with metric-specific routing, graph analysis, API validation, scenario management, benchmarking and analytics. Source inspection and local tests reproduced the 25-vertex, 102-edge Standard network, its density/connectivity metrics, mode aggregates and CAI-to-MEX cost-optimal path. The repository's 25 automated tests passed. The result is a working instructional platform and a concrete basis for discussing algorithmic assumptions, not an operational logistics system.

The main limitations are synthetic inputs, single-metric optimization, static weights, no live carrier or traffic feeds, no time-dependent paths, no authentication, and machine-sensitive timing results. The reported graph properties describe a generated sample; average delivery values are computed path summaries, not observed service levels. No client, survey, real-user test or production performance finding is asserted.

5.2 Future Work

1. **Dynamic routing:** update weights after closures, incidents or schedules and invalidate/recompute affected paths.
2. **Larger graphs:** profile memory and latency, avoid full graph reconstruction where justified, and evaluate indexed priority queues or specialized preprocessing.
3. **Additional algorithms:** compare A* with an admissible geographic heuristic, Johnson's algorithm for many-source queries with negative edges but no negative cycles, and bidirectional search for suitable non-negative graphs.
4. **Real data integration:** add licensed network, timetable and tariff sources with provenance, freshness timestamps and explicit uncertainty. Do not mix synthetic backhaul credits with physical costs.
5. **Multi-objective planning:** expose Pareto trade-offs or an explicitly normalized weighted objective for cost, distance and time rather than silently combining them.
6. **Time-dependent routing:** represent travel-time functions and departure schedules instead of one static edge time.
7. **Quality and security:** add authentication/authorization, server-side operational controls, privacy review, API abuse limits and threat-informed testing before any deployment with sensitive data.
8. **Experimental rigor:** repeat benchmarks on documented hardware/runtime, add warm-up and variability summaries, and separate algorithm work from API/serialization overhead.

These are proposals, not features currently present in the repository.

REFERENCES

[1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959. doi: 10.1007/BF01386390 (https://doi.org/10.1007/BF01386390).

[2] R. Bellman, “On a routing problem,” *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958. doi: 10.1090/qam/102435 (https://doi.org/10.1090/qam/102435).

[3] L. R. Ford, Jr., *Network Flow Theory*. RAND Corporation Research Memorandum RM-1418, 1956.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.

[5] Python Software Foundation, “`heapq` — Heap queue algorithm,” *Python 3 Library Reference*. [Online]. Available: https://docs.python.org/3/library/heapq.html. Accessed: Nov. 13, 2026.

[6] Pallets Projects, “Welcome to Flask,” *Flask Documentation*. [Online]. Available: https://flask.palletsprojects.com/en/stable/. Accessed: Nov. 13, 2026.

[7] SQLite, “CREATE TABLE,” *SQLite Documentation*. [Online]. Available: https://www.sqlite.org/lang_createtable.html. Accessed: Nov. 13, 2026.

[8] T. Bray, Ed., “The JavaScript Object Notation (JSON) Data Interchange Format,” RFC 8259, Internet Engineering Task Force, Dec. 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8259.

[9] ISO/IEC 25010:2023, *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. International Organization for Standardization / International Electrotechnical Commission, 2023. [Online]. Available: https://www.iso.org/standard/78176.html.

[10] E. F. Moore, “The shortest path through a maze,” in *Proceedings of the International Symposium on the Theory of Switching*, 1959, pp. 285–292.

Algorithm descriptions and complexity statements in this report are paraphrased and cited; the five-chapter organization and visual format are based on the locally supplied Chandigarh University report-format reference, not copied sample prose.

APPENDIX

Appendix 1 – Plagiarism Report

Placeholder for an official similarity report

No similarity scan, percentage, certificate or verification result is represented in this report. Attach the institution-approved plagiarism-check result here after the authorized process has been completed.

Required item	Placeholder
Institution-approved tool	_____
Report date	_____
Similarity percentage	To be completed from the official report
Report/certificate identifier	_____
Authorized reviewer/signature	_____

This placeholder is not a plagiarism certificate and must not be interpreted as a verified similarity outcome.

Appendix 2 – Design Checklist

Table A.1 – Engineering design checklist

Criterion	Evidence in project	Status
Need and scope stated without asserting an invented client	Academic simulator scope and limitations	Complete
Directed graph and edge-weight objectives defined	Graph model uses cost, distance and time	Complete
Algorithm preconditions evaluated	Dijkstra rejects negative selected weights	Complete
Negative-cycle behavior considered	Bellman–Ford detects cycle and affected vertices	Complete
Data structure selection justified	Edge list plus adjacency list; matrix alternative compared	Complete
Data validation and error behavior included	API tests for bounds, duplicates and invalid requests	Complete
Persistence integrity constraints described	SQLite keys, constraints and indexes reviewed	Complete
Verification plan and tests present	25 automated tests passed	Complete
Performance evidence separated from theory	Reported values labelled; local timings separately disclosed	Complete
Real-world dataset and operational claims supported	Synthetic data is explicitly labelled	Complete
Accessibility, privacy and security assessed for production use	Not formally assessed; proposed future work	Not complete
Official supervision/examination information verified	Blank fields retained for institution	Pending official completion
Similarity report attached	Placeholder only	Pending official report

Completion indicators describe documented project evidence; they do not certify compliance with an external standard or institutional approval.

USER MANUAL

The deployed application is available at **https://logistics-platform-t77a.onrender.com/**. The service may take time to wake from an idle state. User-interface behavior below was checked against the local implementation; the deployed Overview rendered after its cold start. No production network data was changed for this report.

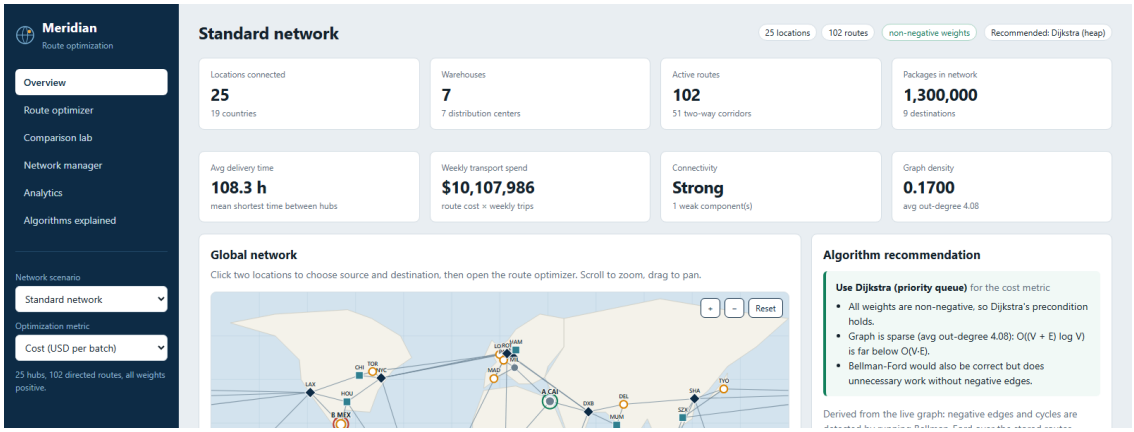


Figure U.1: Meridian local Overview with the Standard scenario selected

Access and basic navigation

1. Open the deployment URL in a modern browser and wait for the service to finish waking. If a loading screen persists, refresh after a short wait.
2. On **Overview**, review the selected network's locations, warehouses, routes, package count, mean shortest delivery time, weekly synthetic spend, connectivity and density. The map is a stylized network view.
3. Use **Network scenario** in the sidebar to select a built-in or generated network. The negative-edge and negative-cycle scenarios are algorithm tests, not realistic tariff data.
4. Use **Optimization metric** to choose cost, delivery time or distance. This changes the edge weights passed to the route calculation; it does not combine the objectives.
5. On the map, click two location markers to choose source and destination. The source/destination indicators update in the view. Scroll/drag behavior is described by the interface; use the route form if map selection is inconvenient.

Calculate and interpret a route

6. Open **Route optimizer**. Confirm the source and destination dropdowns, choose **Auto (recommended)** or an explicit algorithm, and optionally enable alternatives.
7. Select **Calculate route**. If the selected metric has negative weights and a Dijkstra variant is chosen, the app explains why it is blocked; select Bellman–Ford for the negative-edge test scenario. A negative cycle can make the finite optimum undefined.
8. Read the selected-metric total and route path. Review the hop table for mode, distance, cost and estimated time. These are separately accumulated measures; the selected objective determines which one is minimized.
9. If alternatives are enabled and available, review the paths produced by single-link removal. They are not an exhaustive set of all k-shortest paths.

Compare, benchmark and inspect the network

10. Open **Comparison lab**, choose endpoints and metric, then run the algorithm comparison. Applicable algorithms are compared on the same input; review agreement, status, edge checks and medians. Wall-clock values vary and should not be treated as portable guarantees.
11. In the scaling panel, enter/select vertex counts, repeats and average degree where the controls are available, then run the benchmark. The app shows random and Bellman–Ford worst-case generated graphs; benchmark runs do not use live transportation data.
12. Open **Network manager** to inspect network overview and scenario-generation controls. The verified application includes location and route creation/edit/removal, transport modes, route distance/cost/time/trips, bidirectional creation, sample network generation, and reset/delete safeguards. Editing these controls changes the selected database; do not make test edits to a production network.
13. Open **Analytics** to view mode spend/time/cost-per-kilometre, route-efficiency examples, connectivity and degree-ranked hubs. The figures summarize synthetic stored values and path calculations; they are not shipment telemetry.
14. Open **Algorithms explained** to read the application's algorithm guidance, preconditions and complexity comparison.

The deployment may differ from the locally inspected source version. If a control is unavailable or the service is still waking, do not infer successful behavior from the screenshot alone; use the status message and consult the current deployed interface.